

RTL Design Sherpa

Delta Generator Architecture Specification 1.0

October 16, 2026

Table of Contents

1 Delta Gas Index....8
2 Document Information....9
2.1 Delta Generator Architecture Specification....9
2.2 Revision History....10
2.3 Document Purpose....11
2.4 Intended Audience....12
2.5 Related Documents....13
3 What the Generator Produces....14
3.1 Output artifacts....15
3.2 Architecture in the generated flat module....16
3.3 Topology status....17
4 When to Regenerate....18
4.1 Standard regeneration workflow....19
4.2 Pre-generated vs custom....20
5 Generator Entry Points....21
5.1 Flat crossbar generator....21
5.2 Tree primitives generator....22
5.3 Programmatic API....23
6 CLI and Programmatic API....24
6.1 delta_generator.py CLI....24
6.2 complete_tree_generator.py CLI....25
6.3 Programmatic API....27

6.4 Programmatic API for mesh....28
7 Parameter Semantics....29
7.1 Generation-time parameters....29
7.2 Elaboration-time parameters....30
8 AMBA 5, TSTRB, and Parity Options....31
8.1 TSTRB byte qualifiers....31
8.2 AMBA 5 sidebands (--amba5)....32
8.2.1 Parity rule....32
8.2.2 Who generates and who checks....33
8.2.3 Error flags....33
8.2.4 TWAKEUP steering....34
9 Validation Rules....35
9.1 CLI-level checks....36
9.2 Internal checks....37
9.3 Mesh topology validation....38
9.3.1 Header width budget....38
10 Pipeline Overview....40
10.0.1 Figure 3.1: Delta Generator Pipeline....40
10.1 Stages....41
11 Request Decode....42
11.1 Request matrix....43
11.2 Out-of-range TDEST....44
12 Arbitration and Routing....45
12.1 Packet-atomic round-robin arbitration....45

12.2 Crossbar mux..... 46
12.3 Backpressure routing..... 47
13 Boundary Module Instantiation..... 48
13.1 Flat crossbar..... 49
13.2 Tree primitives..... 50
14 Directory Layout..... 51
15 Module Header and Parameters..... 52
15.1 Header comment..... 53
15.2 Port naming convention..... 54
16 Flat Crossbar..... 55
16.1 Internal structures..... 56
16.2 Optional features..... 57
16.2.1 Performance counters..... 57
16.2.2 Assertions..... 57
17 Tree Primitives..... 58
17.1 delta_split_1to2..... 59
17.2 delta_merge_2to1..... 60
17.3 Fan-out / fan-in trees..... 61
17.4 Filelist convention..... 62
18 Full Regeneration Rule..... 63
18.1 Regeneration command..... 64
18.2 Do not hand-edit generated files..... 65
19 Pre-Generated Fixtures..... 66
19.1 Custom variants..... 67

20 DV Suite 68
20.1 Test focus areas 69
20.2 Running the suite 70
20.3 Adding a new test 71
21 Lint Checks 72
21.1 Common concerns 73
21.2 Synthesis smoke test 74
22 Adding a New Topology 75
22.1 Retired tree scaffolding 76
23 Adding a New Generator Option 77
23.1 Backward compatibility 78
23.2 Example: adding a QoS sideband 79
24 Adding AMBA 5 Sidebands 80
24.1 TWAKEUP design note 81
24.2 Check-signal design note 82
25 Testing an Extension 83
25.1 Minimal test plan 84
25.2 Example: adding a custom variant test 85
25.3 Commit discipline 86

List of Figures

Figure 3.1: Delta Generator Pipeline..... 40

List of Tables

Table 1: Document Properties....9
Table 2: Revision History....10
Table 3: Generator Output Artifacts....15
Table 4: delta_generator.py CLI Flags....24
Table 5: complete_tree_generator.py CLI Flags....25
Table 6: Generation-Time Parameters....29
Table 7: Elaboration-Time Parameters....30
Table 8: AMBA 5 check signals emitted by --amba5....32
Table 9: Check-signal ownership per port....33
Table 10: CLI-Level Validation Rules....36
Table 11: Internal Validation Rules....37
Table 12: Mesh Topology Validation Rules....38
Table 13: Implemented Head-Flit Header Fields....38
Table 14: Tree Primitives....50
Table 15: Port Naming Convention....54
Table 16: Flat Crossbar Internal Structures....56
Table 17: Sources That Force Regeneration....63
Table 18: DV Test Scenarios....69
Table 19: Lint Concerns and Mitigations....73

1 Delta Gas Index

Generated: 2026-10-16

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

2 Document Information

2.1 Delta Generator Architecture Specification

Document Properties

Property	Value
Document Title	Delta Generator Architecture Specification
Version	1.0
Date	2026-10-12
Status	Released
Classification	Open Source - MIT License

2.2 Revision History

Revision History

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial GAS release — verified against `delta_generator.py`, `complete_tree_generator.py`, and emitted fixtures in `rtl/`.
1.1	2026-10-12	RTL Design Sherpa	AMBA 5 interface parity per IHI 0051B ch. 5 (issue #102): `TPARITY`/`PW_WIDTH`/`--parity-width` replaced by the odd-parity check signals `TDATACHK` … `TWAKEUPCHK` (spec widths, checked at every port, sticky `*_parity_error`), matching `rtl/amba/axis5`. First `dv/` suite.

2.3 Document Purpose

This Generator Architecture Specification (GAS) documents the Delta AXI-Stream crossbar code generator. It is written for the engineer who must re-run the generator, add a new topology or option, or understand why a generated module looks the way it does.

For the generated hardware, see the **Delta HAS** and **MAS**. For product requirements, see **PRD.md**.

2.4 Intended Audience

- RTL engineers extending the generator
- Verification engineers regenerating fixtures
- Integrators choosing pre-generated vs custom variants

2.5 Related Documents

- **Delta HAS** - Hardware Architecture Specification
- **Delta MAS** - Micro-Architecture Specification
- **PRD.md** - Product requirements
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

3 What the Generator Produces

The Delta generator (`bin/delta_generator.py`) emits a parameterized AXI-Stream flat crossbar module. A companion script (`bin/complete_tree_generator.py`) emits tree-structured primitives (1:2 splitters, 2:1 mergers, fan-out trees, and fan-in trees). All output is single-file SystemVerilog with no post-processing required.

3.1 Output artifacts

Generator Output Artifacts

Artifact	Description
`delta_axis_flat_<M>x<N>.sv`	Core flat crossbar module.
`delta_split_1to2.sv`	1:2 splitter primitive (when `--nodes` is used).
`delta_merge_2to1.sv`	2:1 merger primitive from `complete_tree_generator.py`.
`delta_fanout_1to<N>.sv`	Fan-out tree scaffold (partial implementation).
`delta_fanin_<N>to1.sv`	Fan-in tree scaffold (partial implementation).
`delta_mesh_cell.sv`	Mesh cell module (emitted with `--topology mesh`).
`delta_mesh_fabric_<X>x<Y>x<Z>.sv`	Mesh fabric top that instantiates the cell grid.

3.2 Architecture in the generated flat module

The generated `delta_axis_flat_<M>x<N>` module contains:

1. **Request decode** — `TDEST` is decoded directly into a slave request matrix.
2. **Per-slave arbitration** — round-robin with packet atomicity (grant held until `TLAST`).
3. **Crossbar mux** — routes `TDATA`, `TVALID`, `TLAST`, `TDEST`, `TID`, `TUSER`, optional `TSTRB`, and with `--amba5` the payload check bits.
4. **Optional AMBA 5 logic** — `TWAKEUP` steering, plus the IHI 0051B ch. 5 check signals: payload checks on ingress, control checks generated or checked, and sticky per-port error flags. Built from `rtl/common/dataint_parity`.
5. **Backpressure logic** — `s_axis_tready` driven from `m_axis_tready` of the granted slave.
6. **Optional performance counters** — packet counts per master and per slave.
7. **Optional assertions** — one-hot grant and TDEST-range checks under `FORMAL`.

3.3 Topology status

`--topology` has four conceptual values:

- `flat` (default, implemented) — emits `delta_axis_flat_<M>x<N>.sv`; this is the production artifact.
- `tree` (retired 2026-10-12) — scaffolding only, in `complete_tree_generator.py`; not in regression.
- `mesh` (implemented) — emits `delta_mesh_cell.sv` plus a fabric top. The hierarchy (cell / tile / confluence / fabric) is placement and documentation, not wrapper RTL.

  IMPLEMENTED Mesh mode RTL is generated by `delta_generator.py --topology mesh`. The design is per the owner-approved delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`). The flat crossbar remains the implemented default.

  Still deferred: CXS migration, MonBus observation aggregation in the tile IP, 1 cycle/hop lookahead (issue #120), and SRAM FIFO_IMPL (issue #118).

The flat crossbar and mesh fabric are both generated from the same CLI; the tree scaffolding is retired and remains only for reference.

4 When to Regenerate

Regenerate the Delta fixtures when any of the following change:

- `bin/delta_generator.py`
- `bin/complete_tree_generator.py`
- `bin/rtl_generators/verilog/module.py` (the framework base class)
- The area regression flow or lint rules

4.1 Standard regeneration workflow

```
cd projects/components/fabric-gen-ip/delta/bin
python delta_generator.py --masters 2 --slaves 4 --output-dir ../rtl
python complete_tree_generator.py --type merger --output ../rtl
```

After regeneration, run the DV suite:

```
cd ../dv/tests
make clean-all && make run-all-gate
```

4.2 Pre-generated vs custom

Always check whether a pre-generated fixture exists before generating a custom one. The pre-generated fixtures are the regression baseline; custom variants are for one-off sizes.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

5 Generator Entry Points

5.1 Flat crossbar generator

```
python3 delta_generator.py --masters 2 --slaves 4 --data-width 64 --
output-dir ../rtl
```

CLI flags are documented in the configuration chapter.

5.2 Tree primitives generator

```
python3 complete_tree_generator.py --type merger --output ../rtl
python3 complete_tree_generator.py --type fanout --size 4 --
output ../rtl
python3 complete_tree_generator.py --type fanin --size 4 --output
../rtl
```

5.3 Programmatic API

```
from delta_generator import DeltaConfig, DeltaFlatCrossbar

cfg = DeltaConfig(
    topology="flat",
    num_masters=2,
    num_slaves=4,
    data_width=64,
    dest_width=2,
    id_width=1,
    user_width=1,
    strb_width=0,
    amba5=False,
    registered_outputs=True,
    arbiter_type="round_robin",
    add_performance_counters=True,
)
gen = DeltaFlatCrossbar(cfg)
gen.verilog("../rtl")
```

The `verilog()` method writes `delta_axis_flat_<M>x<N>.sv` into the requested directory using the `Module` framework's `write()` helper.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

6 CLI and Programmatic API

6.1 `delta_generator.py` CLI

```
python3 delta_generator.py --masters 2 --slaves 4 --data-width 64 --
output-dir ../rtl
```

delta_generator.py CLI Flags

Flag	Default	Description
`--topology`	`flat`	`flat` (implemented), `tree` (retired scaffolding), `both` (legacy alias; retired), or `mesh` (implemented).
`--masters`	required	Number of master interfaces.
`--slaves`	required	Number of slave interfaces.
`--data-width`	64	`TDATA` width in bits.
`--user-width`	1	`TUSER` width.
`--strb-width`	0	`TSTRB` width; 0 omits `TSTRB` ports.
`--amba5`	False	Emit AMBA 5 `TWAKEUP`, the IHI 0051B ch. 5 odd-parity check signals and the per-port `*_parity_error` flags. Widths are fixed by the spec; requires a whole-byte `--data-width`.
`--output-dir`	`../rtl`	Destination directory.
`--nodes`	False	Also emit the `delta_split_1to2` primitive.
`--no-counters`	False	Disable the optional performance counters.

6.2 complete_tree_generator.py CLI

complete_tree_generator.py CLI Flags

Flag	Description
`--type`	`merger`, `fanout`, or `fanin`.
`--size`	Number of inputs/outputs for fan-out/fan-in.
`--output`	Output directory (default `../rtl`).

IMPLEMENTED The mesh CLI below is implemented and exercised in regression. It is per the 2026-10-13 mesh design spec.

```
python3 delta_generator.py --topology mesh \
  --cells-x 4 --cells-y 4 --cells-z 1 \
  [--torus-x] [--torus-y] [--torus-z] \
  [--torus x,y,z] \
  [--source-route] [--speculation] \
  [--fifo-impl reg|sram] \
  [--ifc axis|cxs] \
  [--data-width N] [--user-width N] [--amba5] [--no-counters] \
  --output-dir ../rtl
```

Mesh mode emits one cell module, `delta_mesh_cell.sv`, and a fabric top, `delta_mesh_fabric_<X>x<Y>x<Z>.sv`, that instantiates the `cells-x` × `cells-y` × `cells-z` grid. The hierarchy names (cell, tile, confluence, fabric) are for placement and documentation; the generator does not emit wrapper RTL for each level. The `--fifo-impl` knob accepts only `reg` (default) in mesh mode — `sram` is rejected at validation (issue #118: the BRAM registered-read path hangs the drop FSM). `--amba5` is likewise rejected in mesh mode (flat mode only in v1).

`--ifc` selects the interface plane (owner decision 2026-10-16, mirroring the rapids network-plane pattern): `axis` (default) emits the raw AXI4-Stream ports; `cxs` additionally emits `delta_mesh_cell_cxs.sv` (a wrapper around the unchanged AXIS core), four plane modules, and `delta_mesh_fabric_<X>x<Y>x<Z>_cxs.sv` with native AMBA 5 CXS flit ports (Issue D framing, credit grant/return, activation handshake) so a CXS rapids attaches without adapters. CXS mode requires `--data-width 256/512/1024/2048` — AMBA CXS multi-packet flits (`CXSMAXPKTPERFLIT >= 2`, required for TLAST framing) restrict `CXSDATAWIDTH` to those values

(`cxs_pkg::config_error` rule 6); the generator rejects other widths. The §4 header content rides the flit sideband in both modes. Mesh credit sidebands remain the inter-cell flow control; CXS link credit is the plane's port-level mechanism. `axis` remains the default and the checked-in `delta_mesh.f` artifacts are unchanged; the CXS artifacts ship under `delta_mesh_cxs.f`. **Scope:** CXS mode is validated for directed traffic (the dv suite scoreboards self/1-hop/2-hop multi-beat packets end to end); concurrent-load CXS shows a duplication/loss corner tracked as issue #122 — AXIS mode carries concurrent mesh behavior until #122 lands.

`--masters` and `--slaves` are not used in mesh mode; the fabric dimensions define the topology instead.

6.3 Programmatic API

The preferred programmatic entry point is the `DeltaConfig` dataclass plus `DeltaFlatCrossbar`:

```
from delta_generator import DeltaConfig, DeltaFlatCrossbar

cfg = DeltaConfig(
    topology="flat",
    num_masters=2,
    num_slaves=4,
    data_width=64,
    dest_width=2,
    id_width=1,
    user_width=1,
    strb_width=8,
    amba5=True,
    registered_outputs=True,
    arbiter_type="round_robin",
    add_performance_counters=True,
)

gen = DeltaFlatCrossbar(cfg)
gen.verilog("../rtl")
```

6.4 Programmatic API for mesh

```
from delta_mesh import DeltaMeshConfig, generate_mesh, validate_mesh

cfg = DeltaMeshConfig(
    cells_x=4,
    cells_y=4,
    cells_z=1,
    torus_x=False,
    torus_y=False,
    torus_z=False,
    source_route=False,
    speculation=False,
    fifo_impl="reg",
    data_width=64,
    user_width=44,   # must be >= 6*COORD_W + 32; here COORD_W = 2
    amba5=False,
    add_counters=True,
)
validate_mesh(cfg)
generate_mesh(cfg, "../rtl")
```

`dest_width` and `id_width` are normally calculated from `num_slaves` and `num_masters` but may be overridden if the caller supplies them explicitly.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

7 Parameter Semantics

7.1 Generation-time parameters

These are fixed in the emitted module name and structure.

Generation-Time Parameters

Parameter	Effect in emitted RTL
`num_masters`	Number of `s_axis_*` input interfaces; sets `NUM_MASTERS` and `TID` width.
`num_slaves`	Number of `m_axis_*` output interfaces; sets `NUM_SLAVES` and `TDEST` width.
`data_width`	`TDATA` width; sets `DATA_WIDTH`.
`user_width`	`TUSER` width; sets `USER_WIDTH`.
`strb_width`	If > 0, adds `TSTRB` ports and sets `STRB_WIDTH`; otherwise omits them.
`amba5`	Adds `TWAKEUP`, the IHI 0051B ch. 5 check signals and the `s_/m_parity_error` flags; adds the derived check-width parameters.
`registered_outputs`	Reserved for future output-register insertion; currently affects header comments.
`arbiter_type`	Reserved for future arbiter selection; currently only round-robin is emitted.
`add_performance_counters`	Adds 32-bit packet counters per master and slave.

7.2 Elaboration-time parameters

The generated flat crossbar exposes the following parameters:

```
module delta_axis_flat_2x4 #(
    parameter int DATA_WIDTH = 64,
    parameter int DEST_WIDTH = 2,
    parameter int ID_WIDTH   = 1,
    parameter int USER_WIDTH = 1,
    parameter int NUM_MASTERS = 2,
    parameter int NUM_SLAVES  = 4
) (
```

When `strb_width > 0`, `STRB_WIDTH` is added. When `amba5=True`, `TDATACHK_W`, `TIDCHK_W`, `TDESTCHK_W` and `TUSERCHK_W` are added, plus `TSTRBCHK_W` with TSTRB. Each defaults to an expression of its payload width, so leave them at their defaults.

Elaboration-Time Parameters

Parameter	Typical use
`DATA_WIDTH`	Match the datapath width of the surrounding AXI-Stream network.
`DEST_WIDTH`	Wide enough to encode `NUM_SLAVES - 1`.
`ID_WIDTH`	Wide enough to encode `NUM_MASTERS - 1`.
`USER_WIDTH`	Carry sideband metadata alongside each beat.

The master and slave counts are **not** elaboration-time parameters. A 2×4 module cannot be instantiated as 2×8. Generate a new module instead.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

8 AMBA 5, TSTRB, and Parity Options

8.1 TSTRB byte qualifiers

By default Delta omits TSTRB to match the common AXI-Stream use case where every byte is valid. To emit byte-qualifier ports:

```
python3 delta_generator.py --masters 2 --slaves 4 --strb-width 8
```

The generator adds `s_axis_tstrb` and `m_axis_tstrb` arrays and muxes them alongside TDATA.

8.2 AMBA 5 sidebands (`--amba5`)

When `--amba5` is set, the generator adds `TWAKEUP` and the full AXI5-Stream interface-protection set of IHI 0051B chapter 5, `Check_Type = Odd_Parity_Byte_All`. It also adds a sticky error flag on every port. The signal names and widths match `rtl/amba/axis5/*`, so a generated crossbar connects directly to those modules.

`--amba5` needs a whole-byte `--data-width`, because `TDATACHK` has one bit per `TDATA` byte. The CLI rejects anything else.

AMBA 5 check signals emitted by `--amba5`

Check signal	Width (Table 5-1)	Width parameter	Covers
TDATACHK	DATA_WIDTH/8	TDATACHK_W	TDATA
TSTRBCHK	ceil(STRB_WIDTH/8) (only with --strb-width)	TSTRBCHK_W	TSTRB
TLASTCHK	1	–	TLAST
TIDCHK	ceil(ID_WIDTH/8)	TIDCHK_W	TID
TDESTCHK	ceil(DEST_WIDTH/8)	TDESTCHK_W	TDEST
TUSERCHK	ceil(USER_WIDTH/8)	TUSERCHK_W	TUSER
TVALIDCHK	1	–	TVALID
TREADYCHK	1	–	TREADY
TWAKEUPCHK	1	–	TWAKEUP

Each width parameter is derived in the generated module header, for example `parameter int TDATACHK_W = DATA_WIDTH / 8`. If you override a payload width at instantiation, the check ports follow it. There is no separate parity-width option, because the spec fixes the widths. The old `--parity-width` flag has been removed.

8.2.1 Parity rule

Odd parity: a check bit and the bits it covers always hold an odd number of ones. Check bit `[n]` covers bits `[8n+7:8n]`. When a signal is not a whole number of bytes, the top check bit covers the short remainder (IHI 0051B section 5.3). A one-bit check is the inversion of its signal. The crossbar computes and checks every bit with one `rtl/common/dataint_parity` instance (`CHUNKS=1`, `parity_type=0`). The generated file therefore needs `dataint_parity` in its compile closure: `-f $REPO_ROOT/rtl/common/filelists/dataint_parity.f`, as `rtl_test/filelists/delta_axis_flat_2x4_strb_amba5.f` does.

8.2.2 Who generates and who checks

Check-signal ownership per port

Port side	Signal	Crossbar action
master (s_axis_*), crossbar receives	payload *CHK	Checked against the payload while TVALID is high. Then carried to the granted slave port unchanged, by the same grant mux as TDATA.
master	TVALIDCHK, TWAKEUPCHK	Checked every cycle out of reset (check enable is ARESETn).
master	TREADYCHK	Generated from the TREADY the crossbar drives.
slave (m_axis_*), crossbar transmits	payload *CHK	Carried from the granted master. When no master is granted, they idle at all ones, the odd parity of the all-zero idle payload.
slave	TVALIDCHK, TWAKEUPCHK	Generated from the TVALID / TWAKEUP the crossbar drives.
slave	TREADYCHK	Checked every cycle out of reset.

The payload check bits are carried, not regenerated. A check bit that arrives bad at a master port therefore reaches the slave still bad, and both the crossbar and the downstream receiver can see it.

8.2.3 Error flags

s_parity_error[NUM_MASTERS] and m_parity_error[NUM_SLAVES] are sticky and are cleared only by aresetn. A master port's flag sets on a bad payload *CHK beat, a bad TVALIDCHK, or a bad TWAKEUPCHK. A slave port's flag sets on a bad TREADYCHK. The crossbar does not drop or alter a beat that fails its check. The flag reports the failure and the beat is still delivered.

8.2.4 TWAKEUP steering

TWAKEUP is treated as a hint that may arrive before TVALID. The generator decodes it directly from TDEST without waiting for a grant:

```
m_axis_twakeup[s] = OR over all m of (s_axis_twakeup[m] &&
s_axis_tdest[m] == s);
```

m_axis_twakeupchk[s] is generated from this OR.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

9 Validation Rules

The generator validates CLI arguments through `argparse` and additional checks inside `main()`.

9.1 CLI-level checks

CLI-Level Validation Rules

Rule	Behavior
`--topology` must be `flat`, `tree`, `both`, or `mesh`.	Enforced by `choices`.
`--masters` and `--slaves` are required integers.	Enforced by `required=True`.
`--strb-width` must be >= 0.	Custom check; negative values raise an error.
`--amba5` requires `--data-width % 8 == 0`.	Custom check; `TDATACHK` is one bit per `TDATA` byte (IHI 0051B Table 5-1).
`--cells-x`, `--cells-y`, `--cells-z` must be >= 1	Mesh-only; enforced in `validate_mesh`.
A torus wrap on a dimension requires >= 2 cells in that dimension	Mesh-only; enforced in `validate_mesh`.
`--fifo-impl` must be `reg` or `sram`	Mesh-only; enforced in `validate_mesh`.
`--user-width` must be >= the mesh header width	Mesh-only; enforced in `validate_mesh`; header width is `6 * COORD_W + 32`.

9.2 Internal checks

Internal Validation Rules

Rule	Rationale
`num_masters >= 1` and `num_slaves >= 1`	A crossbar needs at least one master and one slave.
`data_width > 0`	Required for meaningful `TDATA`.
`user_width >= 1`	`TUSER` is always at least one bit.
`dest_width = ceil(log2(num_slaves))`	`TDEST` must encode all slave indices.
`id_width = ceil(log2(num_masters))`	`TID` must encode all master indices.

The generator does **not** currently validate that `strb_width` matches `data_width/8`; callers must ensure the width is consistent with their byte-qualifier policy.

9.3 Mesh topology validation

IMPLEMENTED These rules apply to the mesh CLI defined in the 2026-10-13 mesh design spec. They are enforced by `validate_mesh` in `bin/delta_mesh/config.py`.

Mesh Topology Validation Rules

Rule	Rationale
`cells-x >= 1`, `cells-y >= 1`, `cells-z >= 1`	A fabric needs at least one cell in each dimension. `cells-z` is the K-stack height; `K = 1` is a single-layer mesh.
`torus` wrap in a dimension requires `>= 2` cells in that dimension	A one-cell dimension cannot form a ring.
`data-width > 0`	Same baseline requirement as flat mode.
`user-width` must fit the mesh flit header	The head-flit `TUSER` carries dest/src coordinates, QoS class, VC, multicast flag, misroute budget, source-route pointer, and packet ID.
`--fifo-impl` must be `reg` or `sram`	Selects the input buffer implementation style. The `sram` choice is accepted by validation but not yet wired to the cell (issue #118).
`--amba5` still requires whole-byte `data-width`	AMBA 5 sidebands carry over unchanged from flat mode.

9.3.1 Header width budget

The implemented flit header uses these fields on the head beat:

Implemented Head-Flit Header Fields

Field	Width
`dst_x`, `dst_y`, `dst_z`	`3 * COORD_W`
`src_x`, `src_y`, `src_z`	`3 * COORD_W`
QoS class	2
multicast	1
VC	1
misroute budget	4
source-route pointer	8

Field	Width
packet ID	16

`COORD_W = max(1, ceil(log2(max(cells-x, cells-y, cells-z))))`. The total header width is `6 * COORD_W + 32`. Validation rejects a `--user-width` smaller than this value. The two VCs and three QoS classes are fixed; the header must encode VC (1 bit) and QoS (2 bits) without truncation.

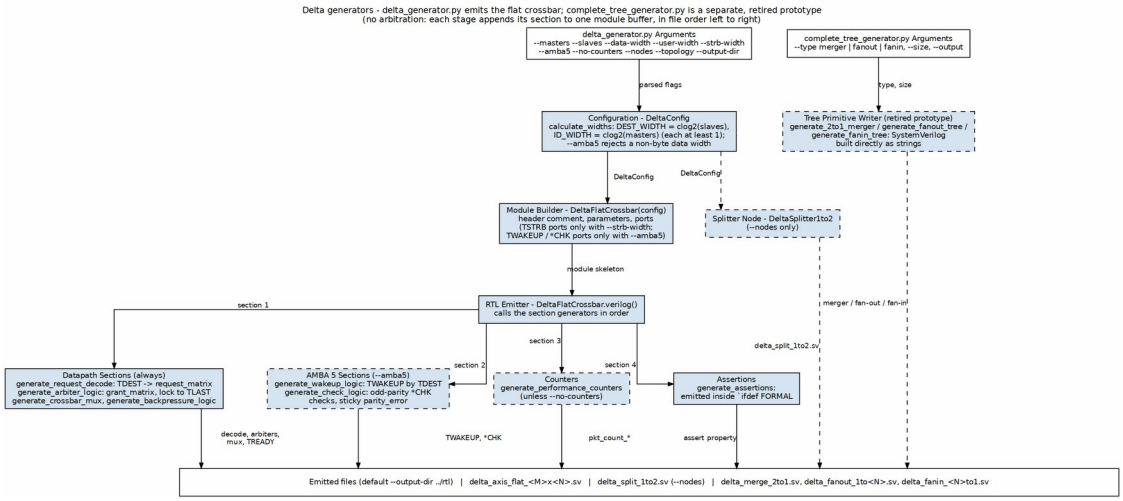
RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

10 Pipeline Overview

The Delta generator uses the `Module` framework from `bin/rtl_generators/verilog/`. It builds the RTL as a sequence of method calls that append comments, instructions, and assignments to an internal buffer, then writes the completed module to disk.

10.0.1 Figure 3.1: Delta Generator Pipeline



Delta Generator Pipeline

The generator parses its flags into a `DeltaConfig`, builds the `DeltaFlatCrossbar` module skeleton, and `verilog()` appends the sections in file order: the always-present datapath, the AMBA 5 sections (`--amba5`), the counters (unless `--no-counters`) and the `FORMAL`-guarded assertions. `--nodes` also writes the 1:2 splitter. `complete_tree_generator.py`, the retired tree prototype, is a separate script that writes its primitives as strings.

10.1 Stages

1. **Argument parsing** — CLI flags become `DeltaConfig` fields.
2. **Width calculation** — `dest_width` and `id_width` are derived from `num_slaves` and `num_masters` unless overridden.
3. **Object construction** — `DeltaFlatCrossbar(config)` declares parameters and ports through the framework.
4. **RTL emission** — `verilog()` calls the generator methods in order:
   - `generate_header_comment()`
   - `generate_request_decode()`
   - `generate_arbiter_logic()`
   - `generate_crossbar_mux()`
   - `generate_backpressure_logic()`
   - `generate_wakeup_logic()` (AMBA 5 only)
   - `generate_check_logic()` (AMBA 5 only: odd-parity `*CHK` checks, sticky `parity_error`)
   - `generate_performance_counters()` (unless disabled)
   - `generate_assertions()`
5. **Framework finalization** — `start()` emits the module header; `end()` emits `endmodule`.
6. **File write** — `write()` saves `delta_axis_flat_<M>x<N>.sv`.

The `complete_tree_generator.py` script is independent and emits splitter/merger/tree scaffolds directly as strings.

11 Request Decode

Delta uses TDEST for routing instead of an address range. This is the fundamental difference between Delta and the APB crossbar generator.

11.1 Request matrix

The generator declares a per-slave request vector:

```
logic [NUM_MASTERS-1:0] request_matrix [NUM_SLAVES];
```

Each entry is populated in an `always_comb` block:

```
for (int s = 0; s < NUM_SLAVES; s++)
    request_matrix[s] = '0;

for (int m = 0; m < NUM_MASTERS; m++)
    if (s_axis_tvalid[m] && (s_axis_tdest[m] < NUM_SLAVES))
        request_matrix[s_axis_tdest[m]][m] = 1'b1;
```

Only masters with a valid beat and a `TDEST` inside the supported slave range contribute to the request matrix.

11.2 Out-of-range TDEST

The generator includes a formal assertion that flags `TDEST >= NUM_SLAVES`. The behavior in simulation for an out-of-range `TDEST` is undefined; the request is simply dropped. System-level validation must ensure that masters never present an invalid destination.

---

12 Arbitration and Routing

12.1 Packet-atomic round-robin arbitration

Unlike the APB crossbar, which re-arbitrates every cycle, Delta locks the grant until the end of the current packet. This prevents interleaving beats from different masters on the same slave.

```
logic [NUM_MASTERS-1:0] grant_matrix [NUM_SLAVES];
logic [$clog2(NUM_MASTERS)-1:0] last_grant [NUM_SLAVES];
logic packet_active [NUM_SLAVES];
```

Arbitration state machine for each slave:

1. If `packet_active[s]` is true, hold the grant until `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]`.
2. If no packet is active and `request_matrix[s]` is non-zero, choose the next requesting master in round-robin order after `last_grant[s]`.
3. Set `packet_active[s]` when a new grant is awarded.

12.2 Crossbar mux

A per-slave `always_comb` block muxes the granted master's AXI-Stream signals to the slave output:

```
for (int m = 0; m < NUM_MASTERS; m++)
    if (grant_matrix[s][m]) begin
        m_axis_tdata[s]  = s_axis_tdata[m];
        m_axis_tvalid[s] = s_axis_tvalid[m];
        m_axis_tlast[s]  = s_axis_tlast[m];
        m_axis_tdest[s]  = s_axis_tdest[m];
        m_axis_tid[s]    = s_axis_tid[m];
        m_axis_tuser[s]  = s_axis_tuser[m];
        // TSTRB included when enabled; with --amba5 the payload check
        // bits (TDATACHK..TUSERCHK) ride the same grant, carried
unchanged
    end
```

12.3 Backpressure routing

Each master's `s_axis_tready` is driven from the `m_axis_tready` of the slave to which it is currently granted:

```
for (int m = 0; m < NUM_MASTERS; m++) begin
    s_axis_tready[m] = 1'b0;
    for (int s = 0; s < NUM_SLAVES; s++)
        if (grant_matrix[s][m])
            s_axis_tready[m] = m_axis_tready[s];
end
```

This preserves AXI-Stream ready/valid handshake semantics across the switch.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

13 Boundary Module Instantiation

Delta does not instantiate external boundary modules for the flat crossbar. The `DeltaFlatCrossbar` class directly emits the full crossbar logic using the `Module` framework. However, the generator does produce small protocol-agnostic primitives when tree generation is requested.

13.1 Flat crossbar

The entire crossbar is self-contained in `delta_axis_flat_<M>x<N>.sv`:

- No external arbiter module is instantiated; arbitration is emitted inline.
- No external FIFOs are used; the design is purely combinational between registered arbitration state.

13.2 Tree primitives

When `--nodes` is passed, `delta_split_1to2.sv` is emitted by the same generator. The `complete_tree_generator.py` script emits additional primitives:

Tree Primitives

Primitive	Purpose
`delta_split_1to2`	Routes one AXI-Stream input to two outputs based on a `TDEST` bit.
`delta_merge_2to1`	Arbiters two inputs to one output with packet atomicity.
`delta_fanout_1to<N>`	Schematic fan-out tree using splitters (partial implementation).
`delta_fanin_<N>to1`	Schematic fan-in tree using mergers (partial implementation).

The tree primitives are not yet production-complete; they exist as building blocks and documentation of the intended topology.

14 Directory Layout

```
delta/
├── bin/
│   ├── delta_generator.py            # framework-based flat crossbar
generator
│   ├── complete_tree_generator.py    # tree primitive generator
│   ├── delta_generator_v1_backup.py  # legacy generator (reference
only)
│   └── delta_performance_model.py    # performance estimation tool
├── rtl/
│   └── delta_axis_flat_<M>x<N>.sv    # generated flat crossbar
├── dv/
│   └── tests/                        # cocotb test suite
├── PRD.md                            # product requirements
└── CLAUDE.md                         # AI development guide
```

The generator writes directly into the directory specified by `--output-dir` (default `../rtl`). Unlike the APB crossbar, Delta does not currently maintain a batch script with a fixed variant table; callers generate the sizes they need.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

15 Module Header and Parameters

The `Module` framework emits the module header automatically from the parameter and port strings added by `DeltaFlatCrossbar.__init__`.

15.1 Header comment

The generated file begins with a block comment documenting the configuration:

```
//
================================================================================
========
// Module: delta_axis_flat_2x4
// Project: Delta - AXI-Stream Crossbar Generator
//================================================================================
==========
// Description: AXI-Stream 2×4 Flat Crossbar Switch
//
// Configuration:
//   - Topology: flat
//   - Masters: 2
//   - Slaves: 4
//   - Data Width: 64 bits
//   - TDEST Width: 2 bits
//   - TID Width: 1 bits
//   - Arbiter: round_robin
//
// Features:
//   - Per-slave round-robin arbitration
//   - Packet atomicity (locked grant until TLAST)
//   - Registered outputs for timing closure
//   - Full AXI-Stream compliance
//================================================================================
==========
```

15.2 Port naming convention

Port Naming Convention

Direction	Naming
Master-side inputs	`s_axis_<signal>[NUM_MASTERS]`
Master-side outputs	`s_axis_tready[NUM_MASTERS]`
Slave-side outputs	`m_axis_<signal>[NUM_SLAVES]`
Slave-side inputs	`m_axis_tready[NUM_SLAVES]`

Optional signals (`tstrb`, `twakeup`, the `t*chk` check signals and the `s_/m_parity_error` flags) are declared only when enabled. With `--amba5` the header also gains derived check-width parameters (`TDATACHK_W = DATA_WIDTH / 8` and so on), and the module instantiates `rtl/common/dataint_parity`.

16 Flat Crossbar

The flat crossbar is the production artifact. It is a single module that connects any master to any slave based on TDEST.

16.1 Internal structures

Flat Crossbar Internal Structures

Signal / Variable	Purpose
`request_matrix[NUM_SLAVES]`	One-hot request vector per slave.
`grant_matrix[NUM_SLAVES]`	One-hot grant vector per slave.
`last_grant[NUM_SLAVES]`	Round-robin pointer per slave.
`packet_active[NUM_SLAVES]`	Lock grant until `TLAST`.

16.2 Optional features

16.2.1 Performance counters

When `add_performance_counters=True`, the generator emits 32-bit packet counters:

```
logic [31:0] pkt_count_master [NUM_MASTERS];
logic [31:0] pkt_count_slave [NUM_SLAVES];
```

Counters increment on `valid && ready && tlast` at the master or slave interface.

16.2.2 Assertions

Under `` `ifdef FORMAL ``:

- One-hot grant check per slave.
- TDEST range check per master.

These assertions are intended for formal verification flows and are stripped during normal simulation if `FORMAL` is not defined.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

17 Tree Primitives

> **Note** Tree generation was retired on 2026-10-12. The `delta_merge_2to1`, `delta_fanout_1toN`, and `delta_fanin_Nto1` scaffolding remains in the tree for reference but is not part of the regression baseline. Hierarchy naming going forward follows the mesh design spec: cell / tile / confluence / fabric.

Delta includes scaffolding for tree topologies in addition to the flat crossbar. These primitives are generated separately and are intended for future hierarchical switches.

17.1 delta_split_1to2

Generated by `delta_generator.py --nodes`. It routes one AXI-Stream input to two outputs based on a configurable TDEST bit:

```
assign route_to_1 = s_axis_tdest[SPLIT_BIT];
assign m0_axis_tvalid = s_axis_tvalid && !route_to_1;
assign m1_axis_tvalid = s_axis_tvalid && route_to_1;
assign s_axis_tready  = route_to_1 ? m1_axis_tready : m0_axis_tready;
```

17.2 delta_merge_2to1

Generated by `complete_tree_generator.py --type merger`. It arbitrates two inputs to one output using round-robin with packet atomicity.

17.3 Fan-out / fan-in trees

`complete_tree_generator.py --type fanout --size N` and `--type fanin --size N` emit structural scaffolds for cascaded splitters and mergers. The implementations are currently partial; they illustrate the intended topology and are not yet part of the regression baseline.

17.4 Filelist convention

A complete Delta subsystem typically needs only:

`rtl/delta_axis_flat_<M>x<N>.sv`

Tree-based systems add the appropriate splitter/merger files. The component-level `Makefile` or area flow maintains the canonical filelist.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

18 Full Regeneration Rule

Any change to the files listed below must be followed by regeneration of the affected fixtures and a full regression run.

Sources That Force Regeneration

Source	Why it forces regeneration
`bin/delta_generator.py`	All flat crossbar output comes from this file.
`bin/complete_tree_generator.py`	All tree primitive output comes from this file.
`bin/rtl_generators/verilog/module.py`	Framework changes affect header/footer formatting.

18.1 Regeneration command

```
cd projects/components/fabric-gen-ip/delta/bin
python delta_generator.py --masters 2 --slaves 4 --output-dir ../rtl
python complete_tree_generator.py --type merger --output ../rtl
```

After regeneration, the only expected `git diff` should be in `rtl/`.

18.2 Do not hand-edit generated files

The `.sv` files in `rtl/` are generator output. Hand-edits will be overwritten on the next regeneration. Fix the generator instead.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

19 Pre-Generated Fixtures

Delta does not currently ship a fixed family of pre-generated sizes. The recommended baseline fixtures are generated on demand with:

```
python delta_generator.py --masters 2 --slaves 4 --data-width 64 --
output-dir ../rtl
python delta_generator.py --masters 4 --slaves 4 --data-width 64 --
output-dir ../rtl
python delta_generator.py --masters 2 --slaves 4 --data-width 64 --
amba5 --output-dir ../rtl
```

If the project later adopts a canonical fixture set, add a `generate_all.py` batch script and document the fixed sizes here.

19.1 Custom variants

Custom sizes are fully supported. When generating a custom variant, record the exact command in the subsystem integration notes so the fixture can be reproduced.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

20 DV Suite

The component-level DV suite lives in `dv/tests/` and uses cocotb.

20.1 Test focus areas

A complete Delta DV suite should cover:

DV Test Scenarios

Scenario	What to verify
Single-packet routing	A master sends a packet to each slave; `TDEST` selects correctly.
Multi-master contention	Two masters target the same slave; arbitration is fair and packet-atomic.
Backpressure	Assert `m_axis_tready` low; `s_axis_tready` follows correctly.
TLAST handling	Grants release only after `TLAST`.
AMBA 5 sidebands	`TWAKEUP` steers by `TDEST`. Payload `*CHK` is carried with `TDATA`. `TVALIDCHK`/`TWAKEUPCHK`/`TREADYCHK` are the inversions of their signals. A bad check sets the right sticky flag.
Out-of-range TDEST	Formal assertion fires; simulation drops the request.
Performance counters	Counters increment on `valid && ready && tlast`.

20.2 Running the suite

```
cd projects/components/fabric-gen-ip/delta/dv/tests
make clean-all && make run-all-func-parallel
```

Today the suite is one file, `test_delta_axis_flat_2x4_strb_amba5.py`, covering the AMBA 5 row above against `rtl_test/delta_axis_flat_2x4_strb_amba5.sv`. It has three tests at gate, func and full. The other rows are still open for the base flavor.

After a generator change, this must pass with zero failures before the regenerated RTL is committed.

20.3 Adding a new test

1. Add a new cocotb test file in `dv/tests/`.
2. Reference the generated module under test.
3. Add the test name to the area regression list.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

21 Lint Checks

The generated RTL must pass the project’s lint flow.

21.1 Common concerns

Lint Concerns and Mitigations

Concern	Mitigation in generator
Unpacked array lint	Arrays are declared as `logic [W-1:0] name [N]`; ensure the lint tool supports unpacked array ports.
Width mismatches	`DEST_WIDTH` and `ID_WIDTH` are computed from slave/master counts.
Latch inference	All `always_comb` blocks assign all outputs; no incomplete cases.
Uninitialized logic	Default assignments precede conditional overrides in every mux.

21.2 Synthesis smoke test

Run a synthesis tool on the 2×4 variant after any generator change. The expected result is clean elaboration with no combinational loops and no latch inference.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

22 Adding a New Topology

The mesh mode design spec (2026-10-13) is the current reference for adding a topology. The retired tree scaffolding is still in the repo, but it is not a regression baseline and should not be used as a model for new work.

Work through the design spec topics in this order, then land the generator changes:

1. **Topology semantics and routing concept** — read §2 (hierarchy), §5 (routing), and §7 (systolic and pattern support) of the 2026-10-13 mesh design spec. Define the coordinate space, escape/adaptive VC policy, and how standard modes switch.
2. **Generator CLI axis** — read §12. Choose the dimension parameters, optional torus dimensions, source-route and speculation flags, data/user widths, AMBA 5 carry-over, counters, and the FIFO implementation knob.
3. **Emitted artifacts** — read §2.1. Mesh mode emits one cell module (`delta_mesh_cell`) and a fabric top. Cell / tile / confluence / fabric are placement and documentation names, not wrapper RTL levels.
4. **Validation rules** — read §4 (flit format) and §12. Enforce dimension ranges, the torus-wrap minimum, and the `TUSER` width needed for the header fields (dest/src coords, QoS class, VC, multicast flag, misroute budget, source-route pointer, packet ID).
5. **DV suites** — read §13. Plan routing, QoS, multicast, deadlock soak, systolic, counter, DFT, and perf-model calibration suites.

After the design is settled, the generator mechanics are the same as any new mode:

- Add a class in `delta_generator.py` (e.g., `DeltaMeshCrossbar`) and derive it from `Module` or reuse existing building blocks.
- Add the topology name to the `--topology` choices.
- In `main()`, instantiate the new class when the selected topology matches.
- Add a DV test that exercises the new topology.
- Regenerate fixtures and run the full DV suite.

22.1 Retired tree scaffolding

`DeltaSplitter1to2` and `complete_tree_generator.py` remain for reference. A tree crossbar would have recursively instantiated splitters and mergers using `TDEST` bit slices, but that path was retired on 2026-10-12.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

23 Adding a New Generator Option

To add a scalar option:

1. Add the field to `DeltaConfig`.
2. Add the CLI argument to `main()` in `delta_generator.py`.
3. Pass the argument into `DeltaConfig(...)`.
4. Use the config field inside the appropriate `generate_*` method.
5. Document the option in this GAS, the HAS, and the PRD if it changes hardware behavior.
6. Add validation in `main()` or inside the generator method.
7. Add a DV test that exercises the new option.

23.1 Backward compatibility

Default values must preserve existing generated output. If the new option changes the default port list, introduce it as an opt-in flag first.

23.2 Example: adding a QoS sideband

```
@dataclass
class DeltaConfig:
    ...
    qos_width: int = 0
```

Then emit `s_axis_tqos` / `m_axis_tqos` arrays and mux them in `generate_crossbar_mux()`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

24 Adding AMBA 5 Sidebands

The current AMBA 5 support covers TWAKEUP and the full IHI 0051B chapter 5 check-signal set (TDATACHK, TSTRBCHK, TLASTCHK, TIDCHK, TDESTCHK, TUSERCHK, TVALIDCHK, TREADYCHK, TWAKEUPCHK). To add more AMBA 5 sidebands:

1. Add the new signal to `DeltaConfig` (e.g., a width field).
2. In `DeltaFlatCrossbar.__init__`, add the new port declarations when `amba5` is enabled.
3. Add routing logic in a new `generate_*_logic()` method.
4. Call the new method from `verilog()`.
5. If the new signal is covered by a check signal (a new payload field gets a `*CHK` per Table 5-1), add it to `_payload_chk_fields()`. That one list drives the ports, the carry mux, the ingress checkers and the error OR.
6. Update the DV suite (`dv/tests/test_delta_axis_flat_2x4_strb_amba5.py` and its harness) to exercise the new sideband.

24.1 TWAKEUP design note

`TWAKEUP` is a hint, not a data beat. It is routed by `TDEST` without arbitration because it may precede `TVALID`. If a future sideband requires grant-based routing, follow the payload check-bit pattern in `generate_crossbar_mux()`.

24.2 Check-signal design note

The payload check bits ride the same grant as TDATA and are carried, not regenerated, so a beat corrupted upstream still fails downstream. The crossbar also checks them on ingress (enabled by TVALID). It generates the single-bit checks for the signals it drives (TVALIDCHK and TWAKEUPCHK on slave ports, TREADYCHK on master ports) and checks the ones it receives. Every bit is one `rtl/common/dataint_parity` instance, odd parity, CHUNKS=1. A new check must follow the same rule: odd parity, one bit per 8 bits, top bit covers the remainder (IHI 0051B section 5.3).

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

25 Testing an Extension

Every generator change must be exercised by at least one generated fixture and one DV test.

25.1 Minimal test plan

1. **Generator unit test** — call `DeltaFlatCrossbar(cfg).verilog()` with the new option and verify the emitted string contains the expected constructs.
2. **Fixture regeneration** — generate the affected fixtures and inspect the diff.
3. **Simulation test** — add or extend a cocotb test that exercises the new behavior.
4. **Lint/synthesis smoke** — run the lint flow on the affected variant.

25.2 Example: adding a custom variant test

```
# dv/tests/test_delta_flat_4x4.py
import cocotb

class TestDeltaFlat4x4:
    @cocotb.test()
    async def test_basic_routing(self, dut):
        # exercise all 4 masters against all 4 slaves
```

The test must be added to the `run-all-gate` target or an equivalent regression list.

25.3 Commit discipline

Commit generator source changes, regenerated RTL, and DV tests in a single logical change. Do not commit hand-edited generated files.